

CS 380 - GPU and GPGPU Programming

Lecture 8: GPU Architecture, Pt. 6

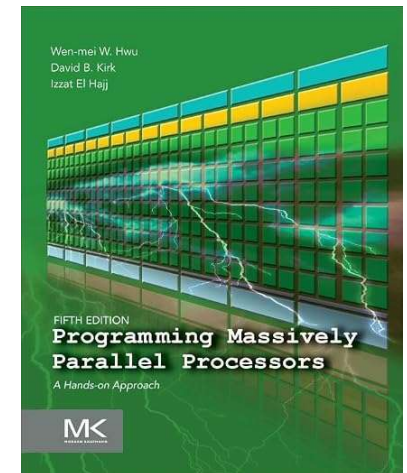
Markus Hadwiger, KAUST

Reading Assignment #5 (until Oct 5)



Read (required):

- Programming Massively Parallel Processors book, 5th edition, **Chapter 3** (*Multi-dimensional grids and data*)
- Programming Massively Parallel Processors book, 5th edition, finish **Chapter 4: 4.4, ...** (*Compute architecture and scheduling*)



- CUDA NVCC documentation [NVCC is the CUDA compiler] (currently CUDA 13.4, Sep 15, 2026):

https://docs.nvidia.com/cuda/pdf/CUDA_Compiler_Driver_NVCC.pdf

Read Chapters 1 – 4, and Chapter 6 (GPU Compilation); get an overview of the rest

Quiz #1: Oct 8



Organization

- First 30 min of lecture
- No material (book, notes, ...) allowed

Content of questions

- Lectures (both actual lectures and slides)
- Reading assignments
- Programming assignments (algorithms, methods)
- Solve short practical examples

Concepts: Latency vs. Throughput



Latency

- What is the time *between start and finish* of an operation/computation?
- How long does it take between starting to execute an instruction until the execution is actually finished / its results are available?
- Examples: 1 FP32 MUL instruction; 1 vertex computation, ...

Throughput

- How many computations (operations/instructions) *finish per time unit*?
- How many instructions of a certain type (e.g., FP32 MUL) finish per time unit (per clock cycle, per second)?

GPUs: ***High-throughput execution*** (at the expense of latency)
(but: *hide* latencies to avoid throughput going down)

Concepts: Types of Parallelism



Instruction level parallelism (ILP)

- In single instruction stream: Can consecutive instructions/operations be executed in parallel? (Because they don't have a dependency)
- Exploit ILP: Execute independent instructions (1) via pipelined execution (instr. pipe), or even (2) in multiple parallel instruction pipelines (superscalar processors)
- On GPUs: also important, but much less than TLP (compare, e.g., Kepler with current GPUs)

Thread level parallelism (TLP)

- Exploit that by definition operations in different threads are independent (if no explicit communication/synchronization is used, which should be minimized)
- Exploit TLP: Execute operations/instructions from multiple threads in parallel (which also needs multiple parallel instruction pipelines)
- **On GPUs: main type of parallelism**

more types:

- Bit-level parallelism (processor word size: 64 bits instead of 32, etc.)
- Data parallelism (SIMD/vector instructions), task parallelism, ...

Concepts: Latency Hiding



**Not about latency of single operation or group of operations:
It's about avoiding that the *throughput* goes below peak**

Hide latency that *does* occur for one instruction (group) by
executing a different instruction (group) as soon as current one stalls:

→ *Total throughput does not go down*

In GPUs, hide latencies via:

- **TLP: pull independent, not-stalling instruction from other thread group**
- ILP: pull independent instruction from down the inst. stream in same thread group
- Depending on GPU: TLP often sufficient, but sometimes also need ILP
- However: If in one cycle TLP doesn't work, ILP can jump in or vice versa

GPU Architecture

Big Idea #3

Next Problem: Stalls!

Stalls occur when a core cannot run the next instruction because of a dependency on a previous operation.

Texture access latency = 100's to 1000's of cycles
(also: instruction pipelining hazards, ...)

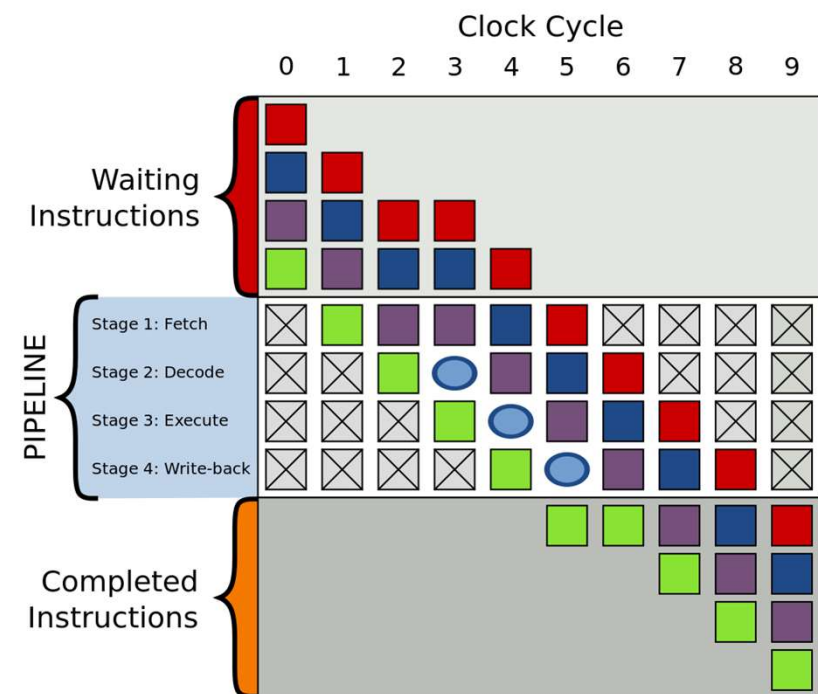
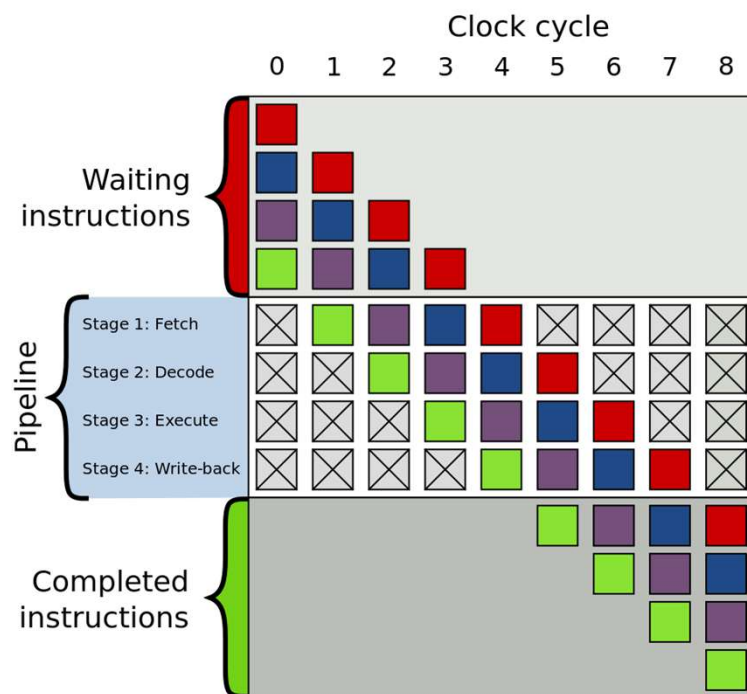
We've removed the fancy caches and logic that helps avoid stalls.

Interlude: Instruction Pipelining



Most common way to exploit *instruction-level parallelism* (ILP)

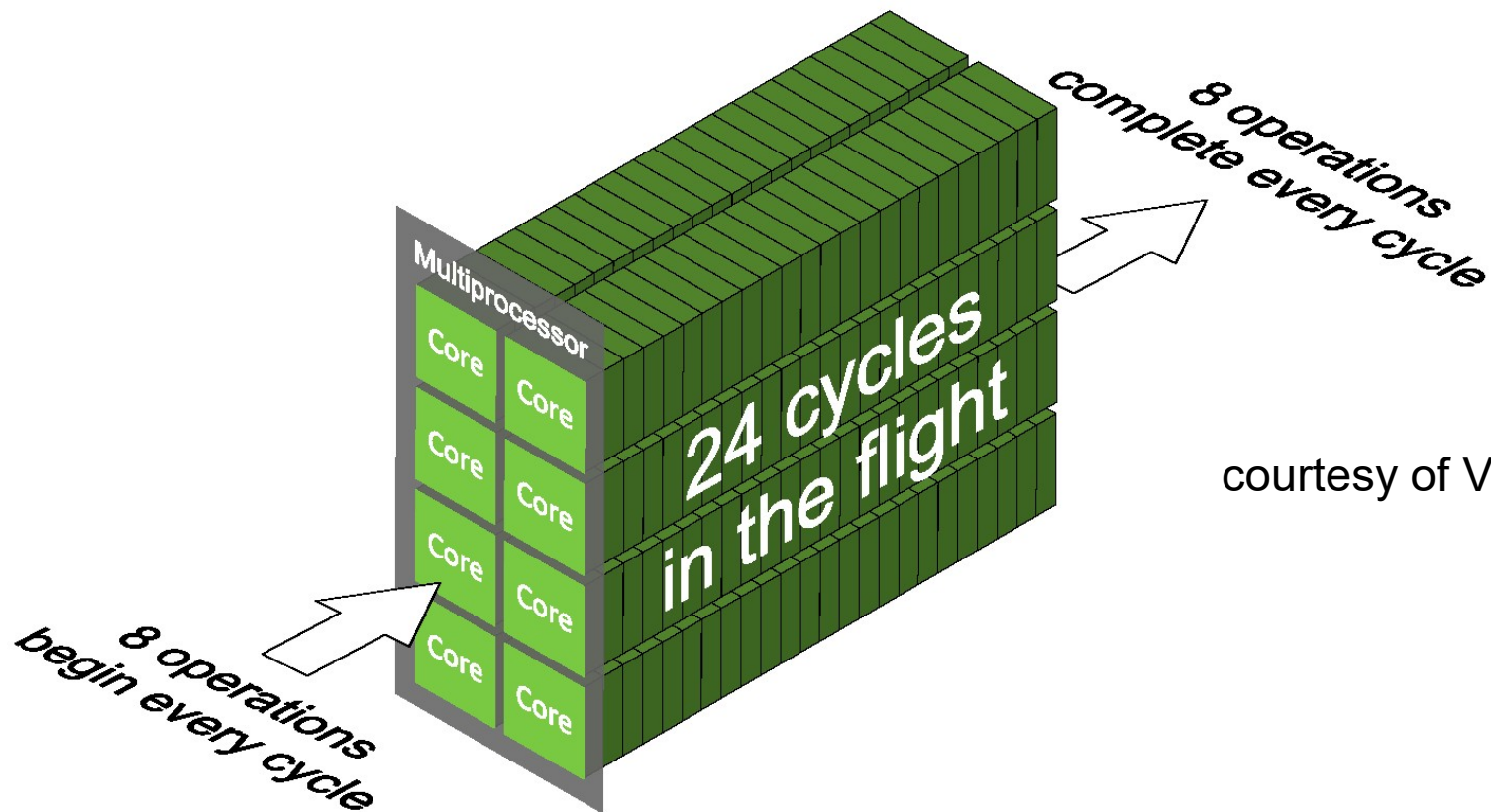
Problem: hazards (different solutions: bubbles, forwarding, ...)



wikipedia

https://en.wikipedia.org/wiki/Instruction_pipelining
https://en.wikipedia.org/wiki/Classic_RISC_pipeline

Interlude: Use Little's law



courtesy of Vasily Volkov

Needed parallelism = **Latency x Throughput**

If we ignore ILP, this gives the *minimum* number of required threads^(*) to keep the cores (ALUs) busy... even without any extra desired latency hiding ability!

(*) but we actually then do this with warps!

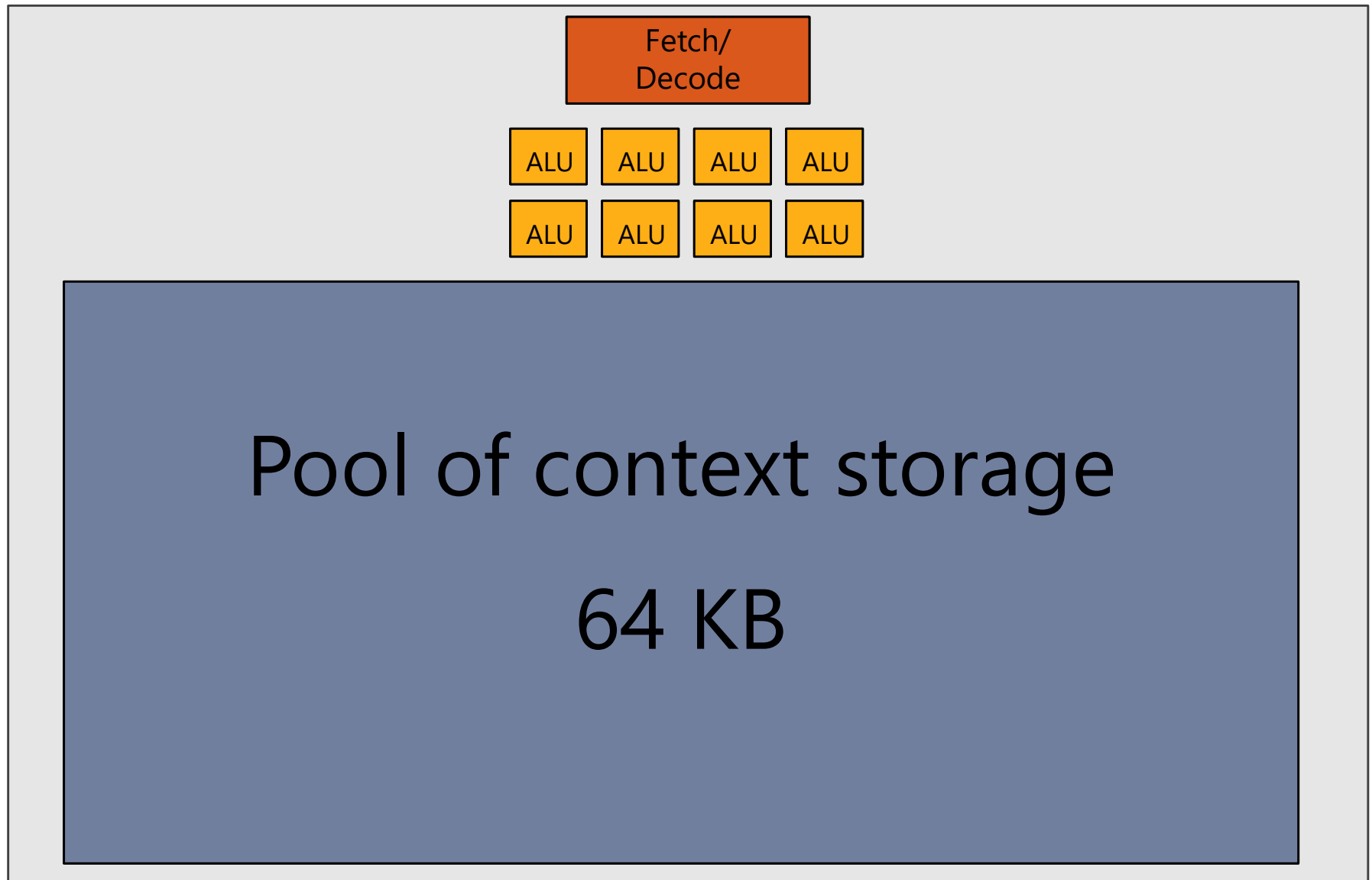
Idea #3: Interleave execution of groups

But we have **LOTS** of independent fragments.

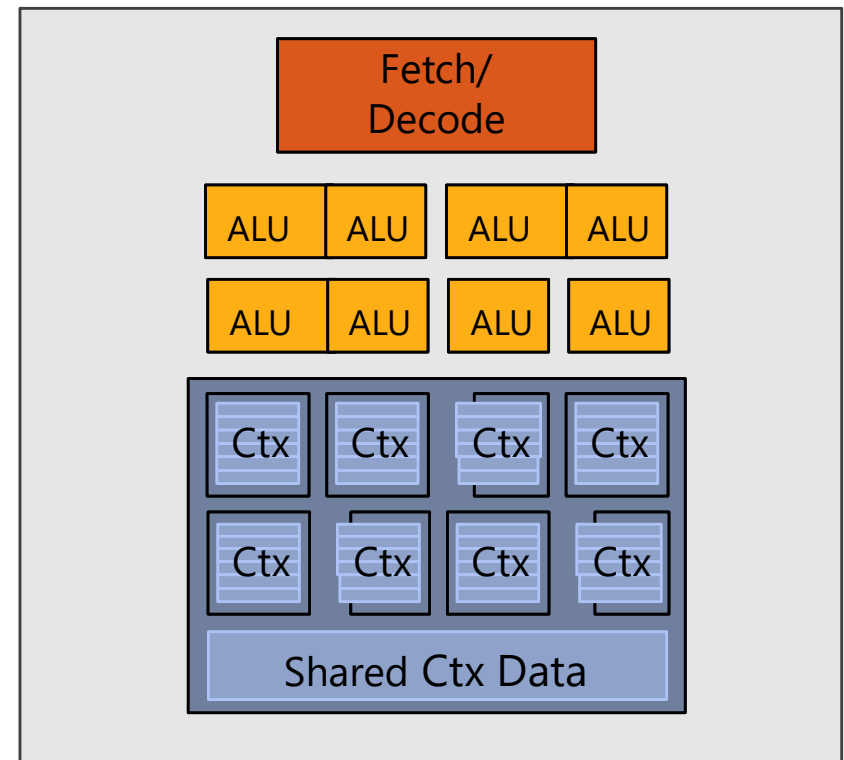
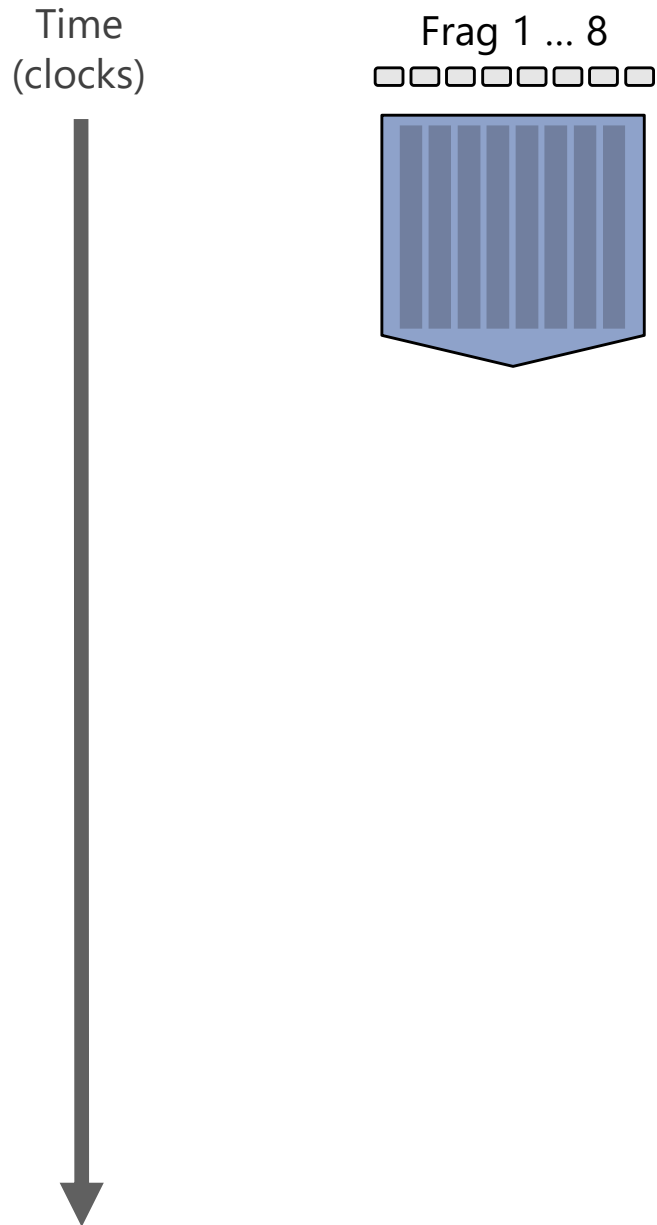
Idea #3:

Interleave processing of many fragments on a single core to avoid stalls caused by high latency operations.

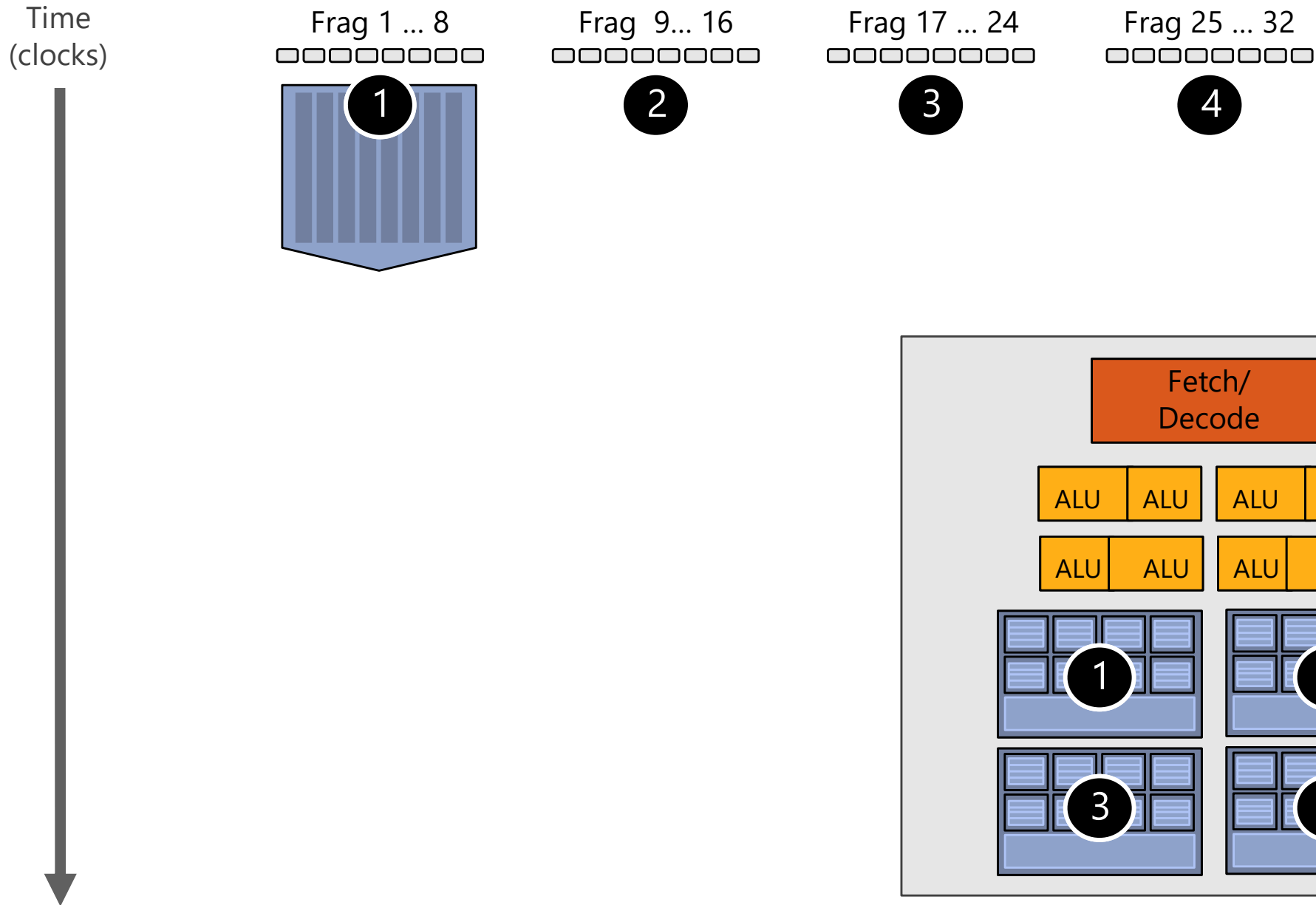
Idea #3: Store multiple group contexts



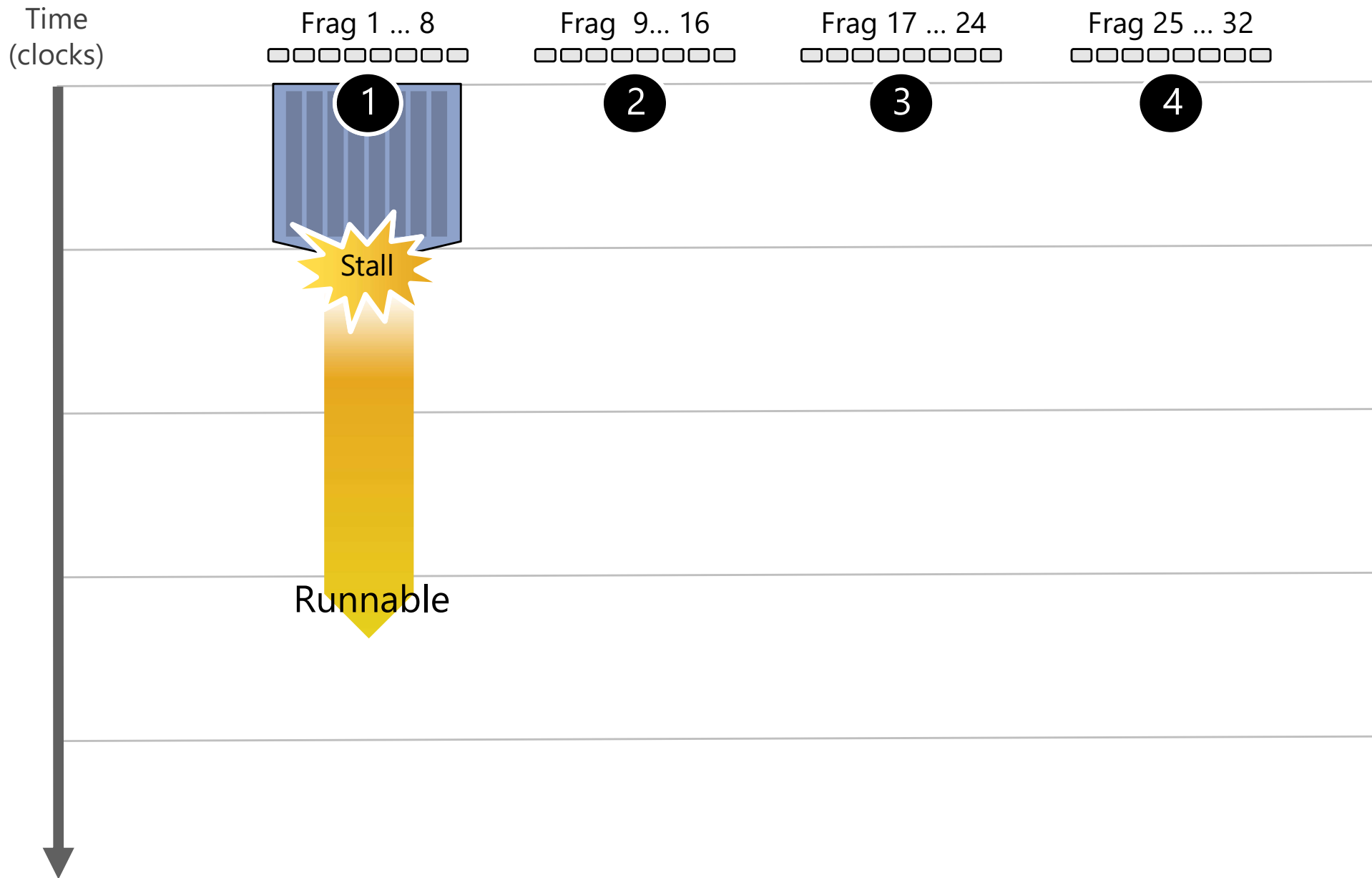
Hiding shader / kernel stalls



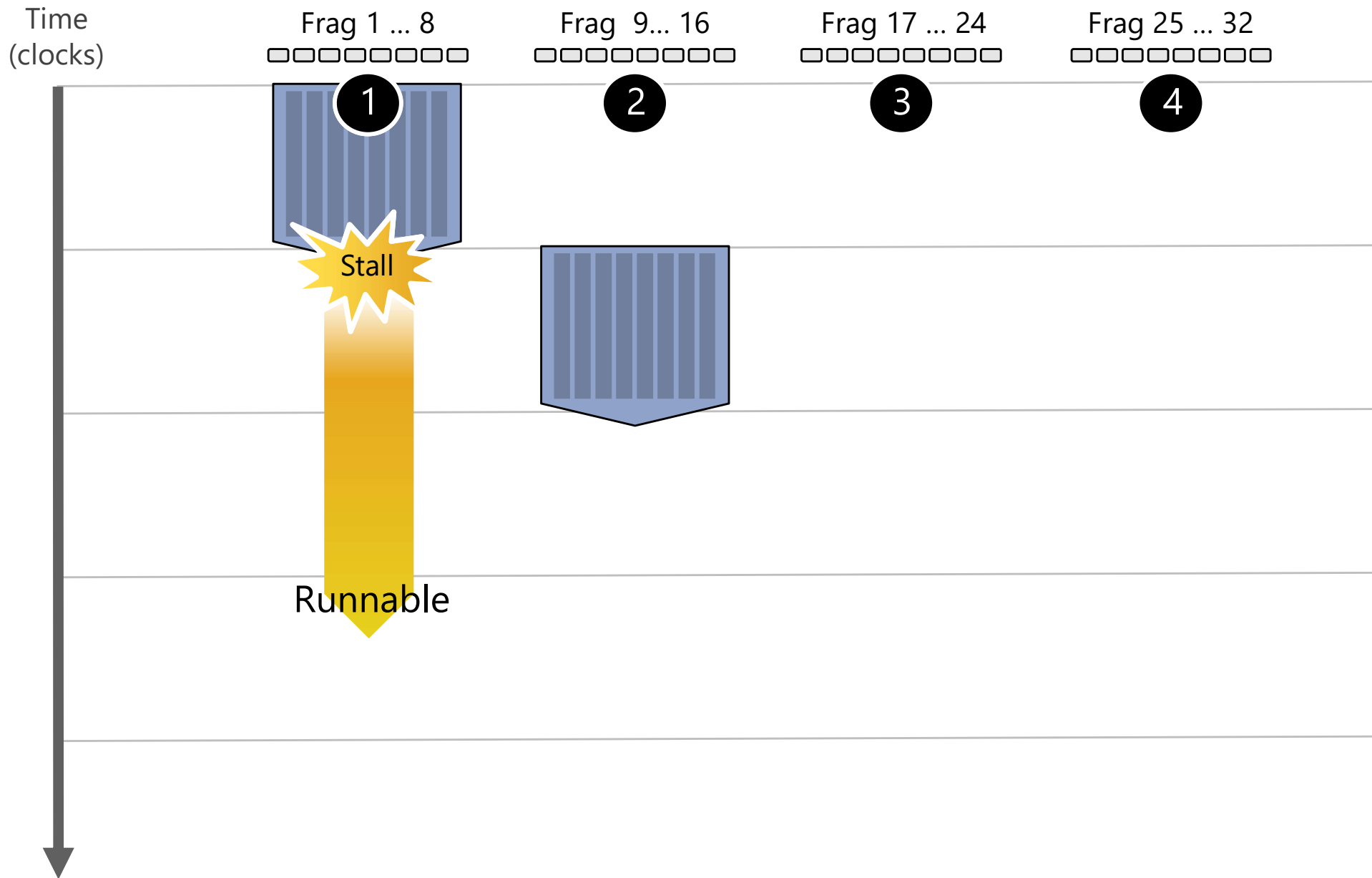
Hiding shader / kernel stalls



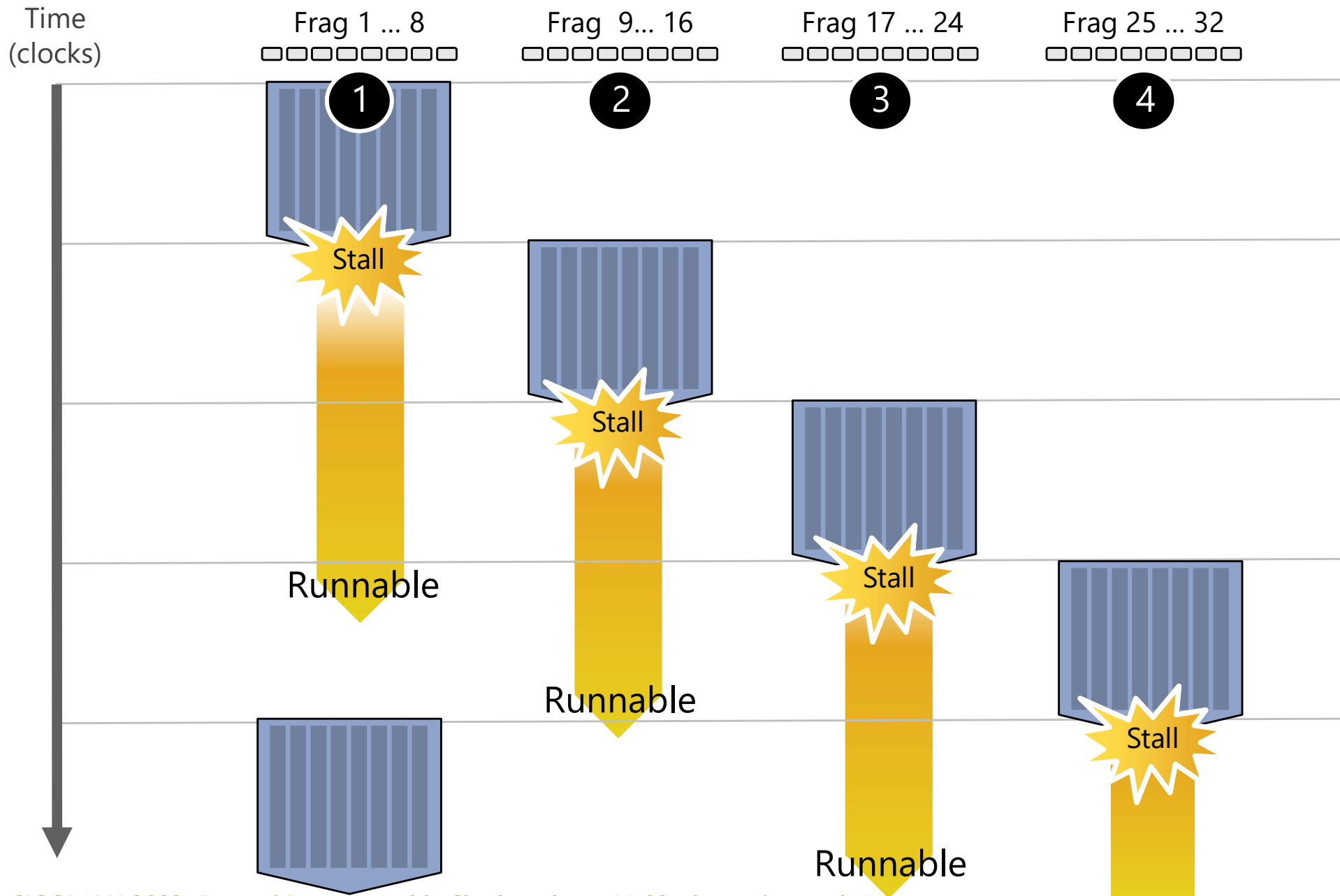
Hiding shader / kernel stalls



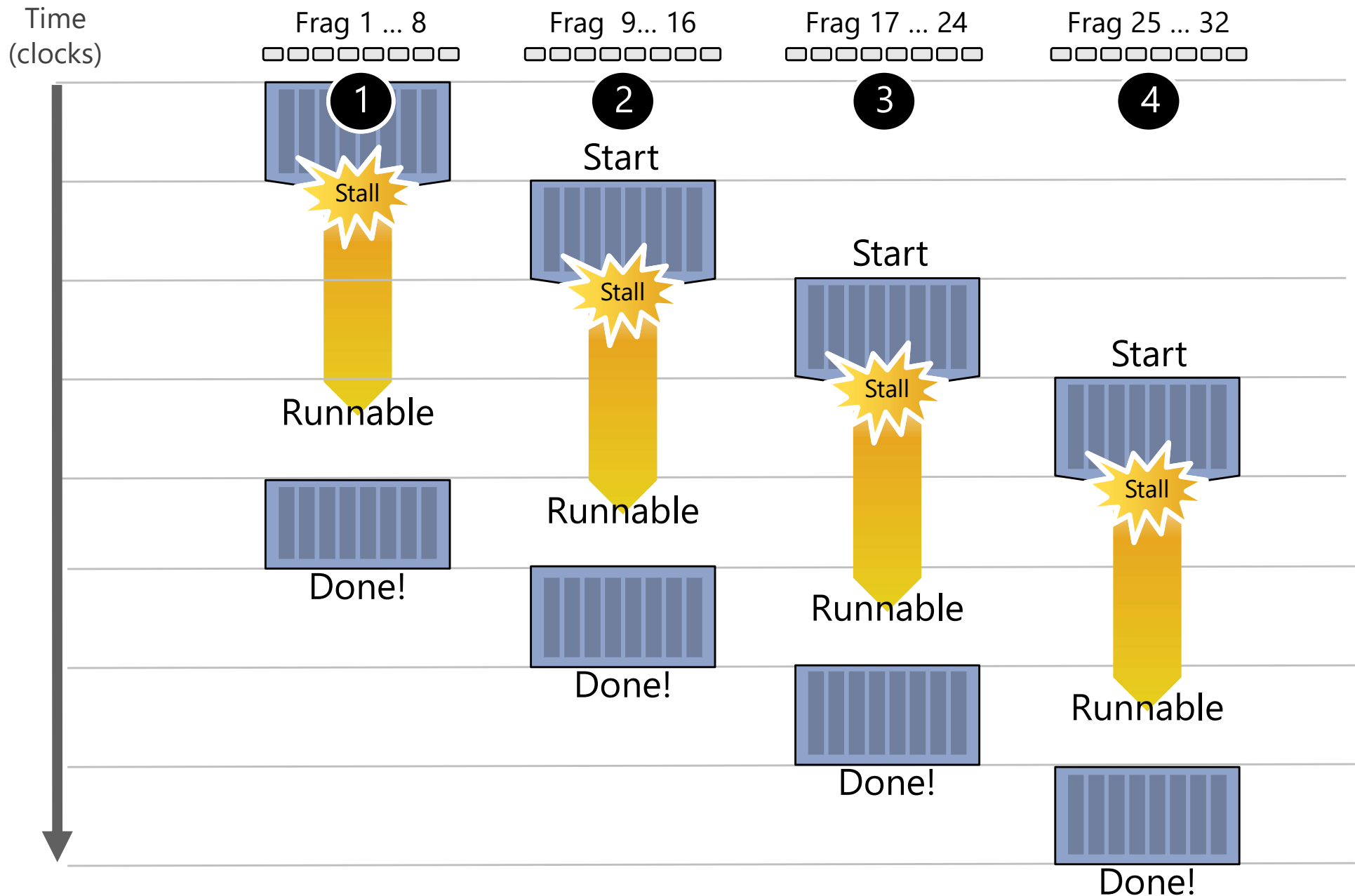
Hiding shader / kernel stalls



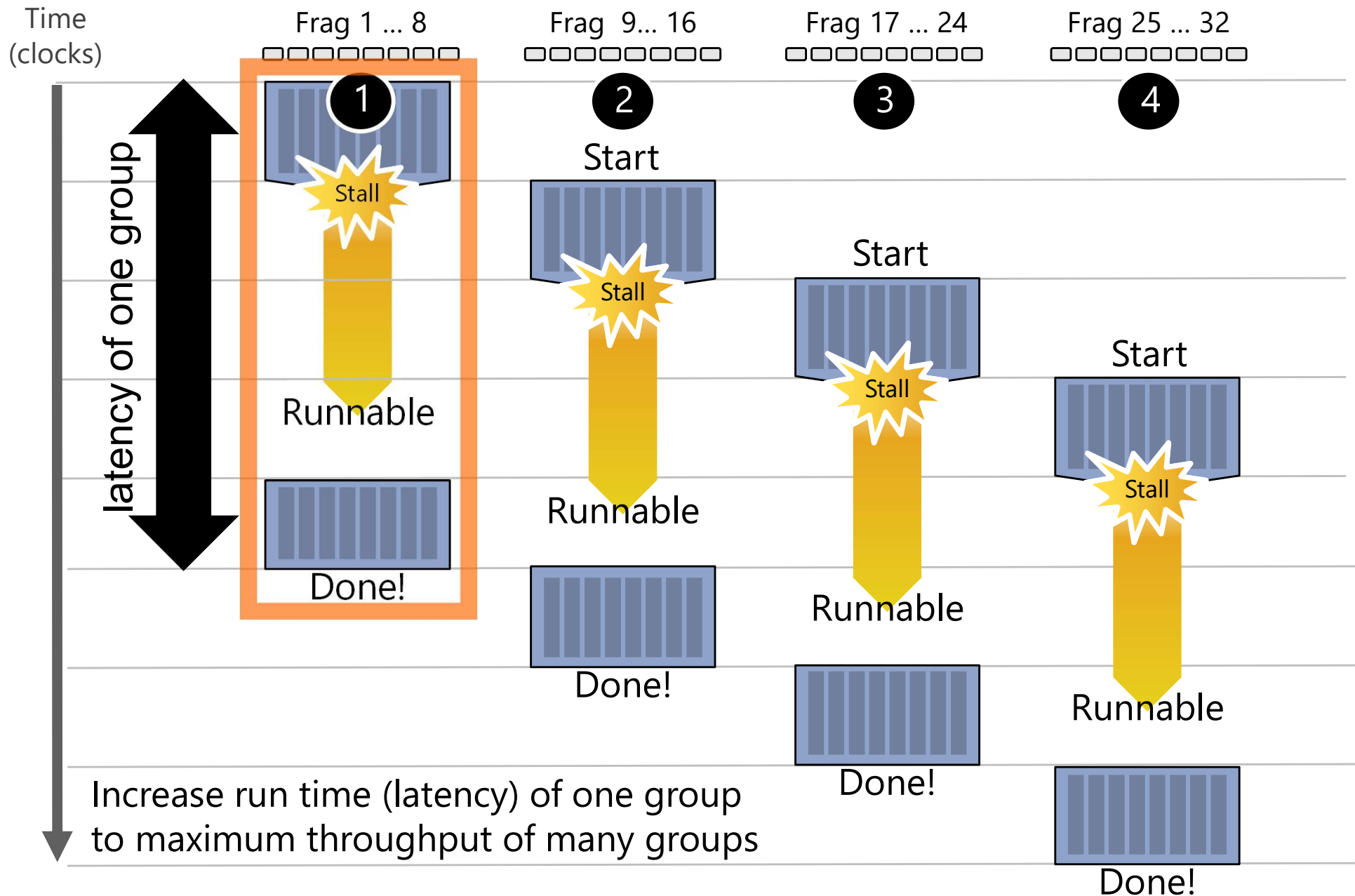
Hiding shader / kernel stalls



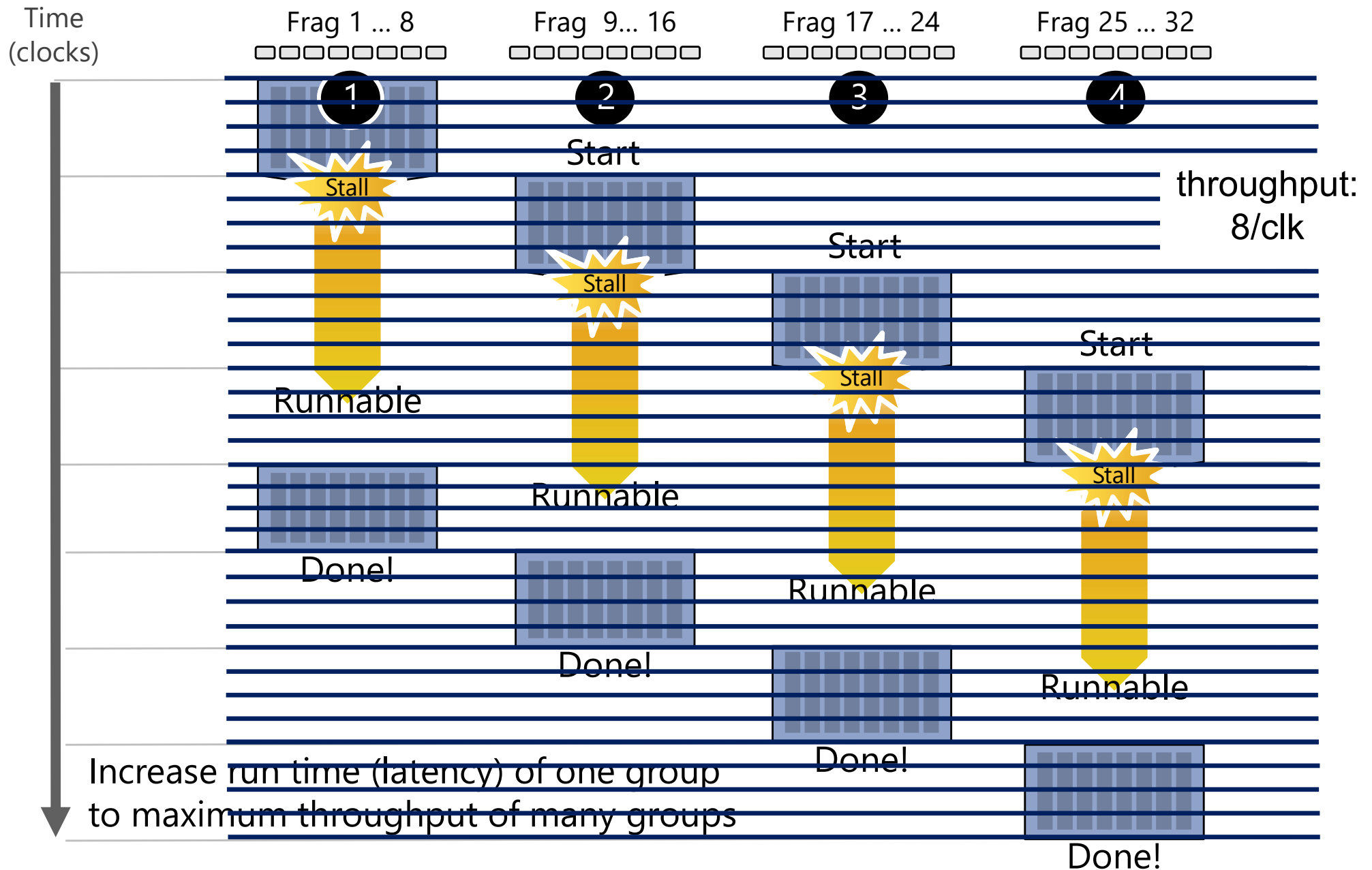
Hiding shader / kernel stalls



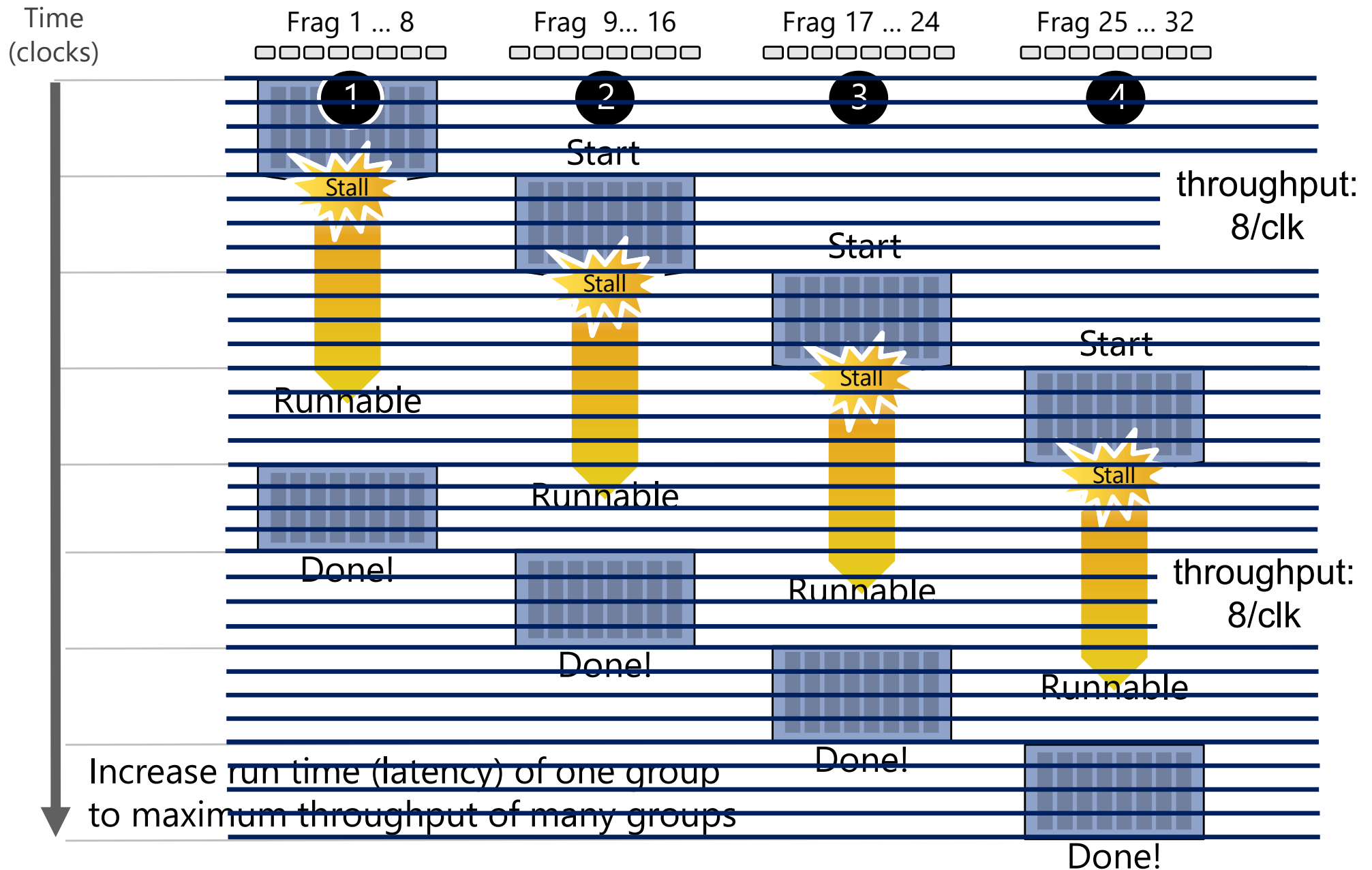
Hiding shader / kernel stalls



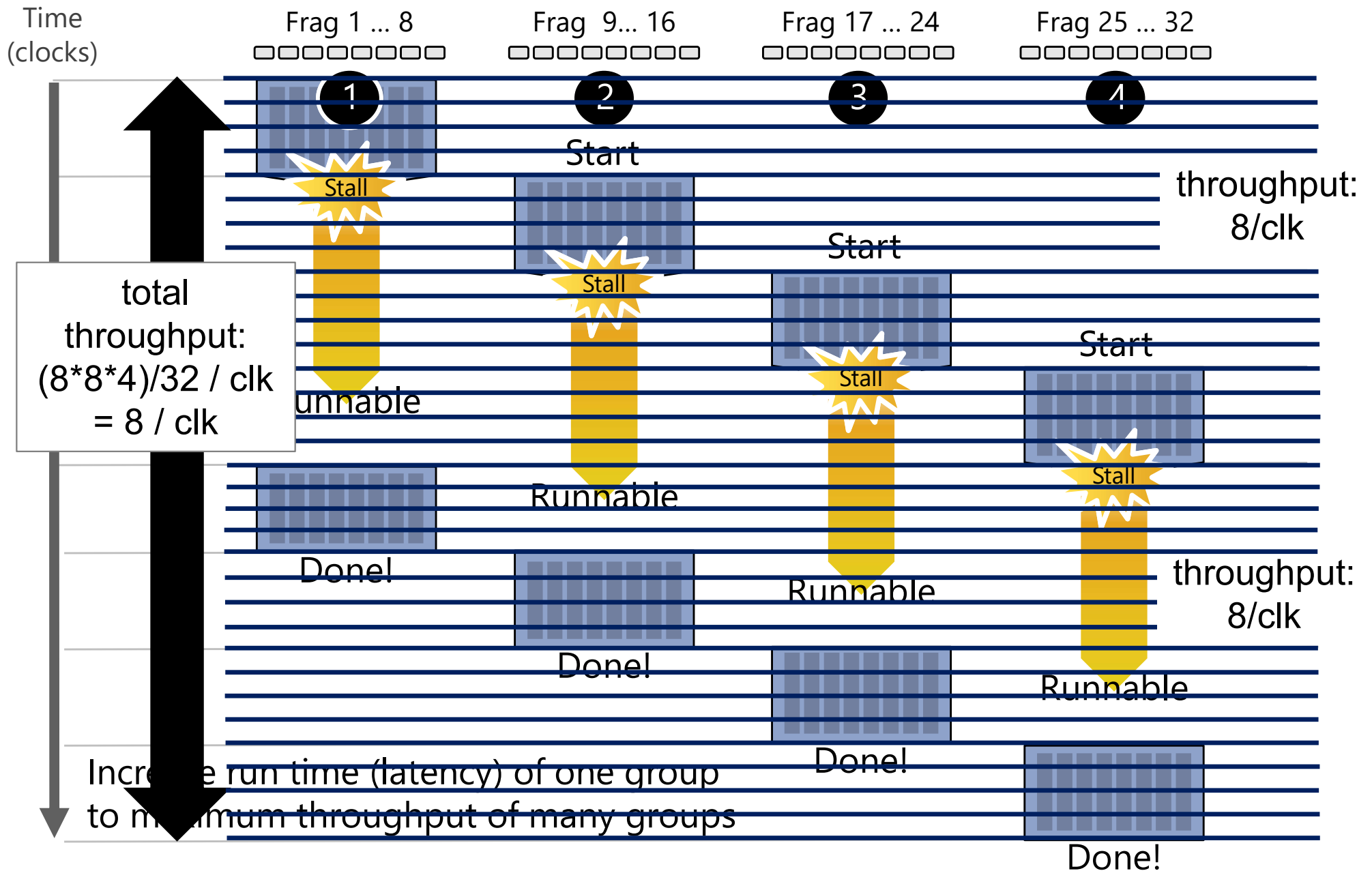
Throughput! (4 groups of threads)



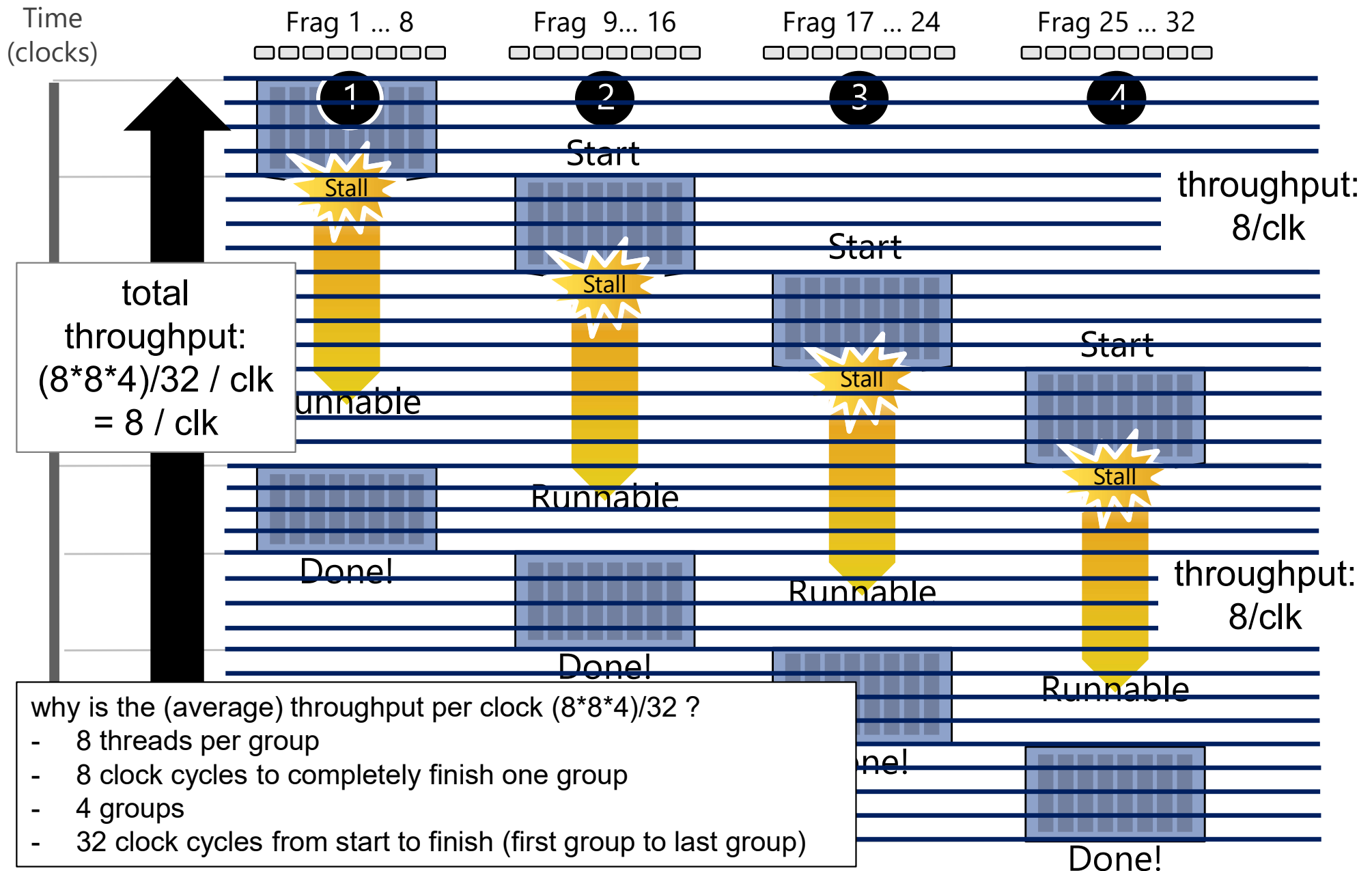
Throughput! (4 groups of threads)



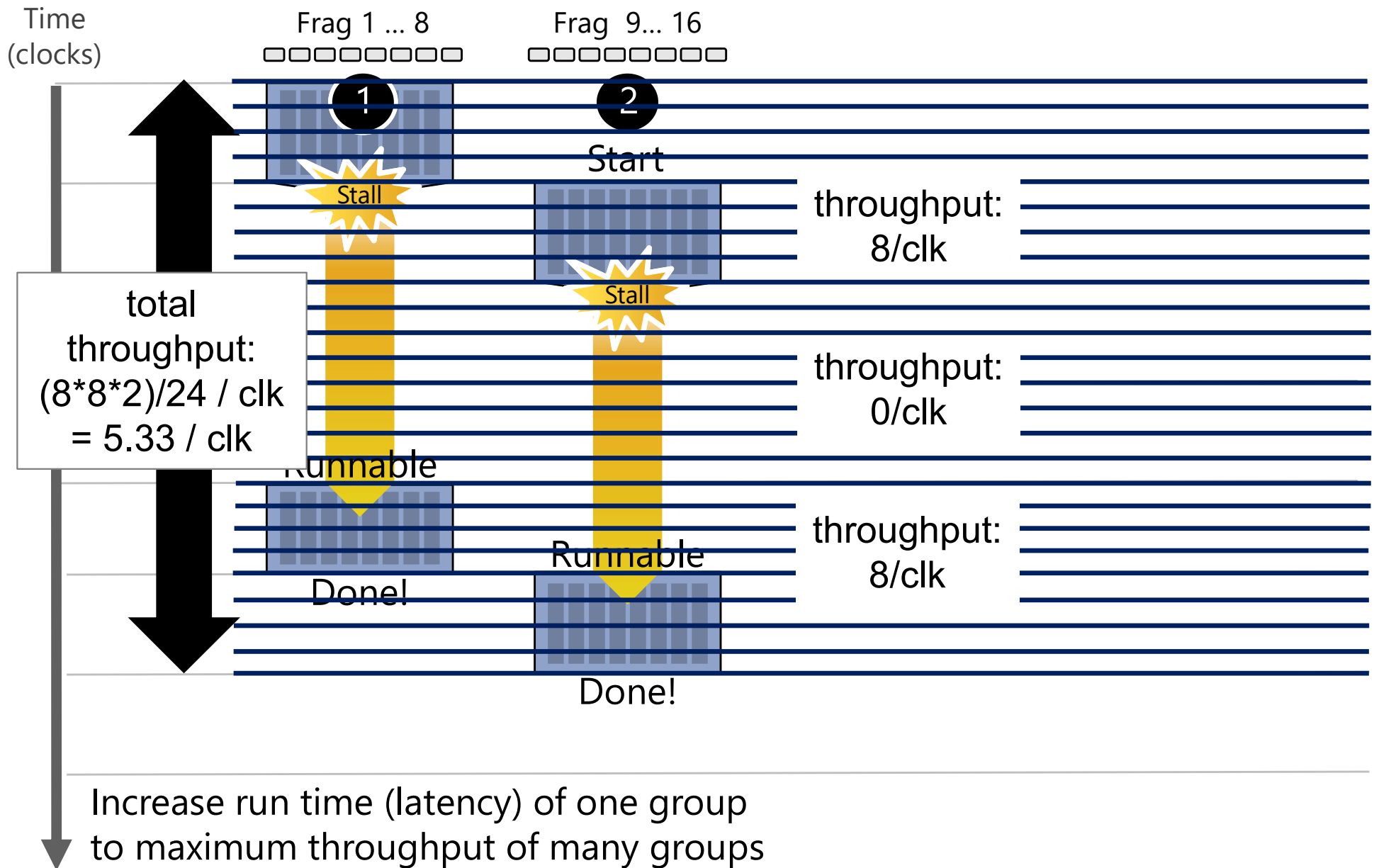
Throughput! (4 groups of threads)



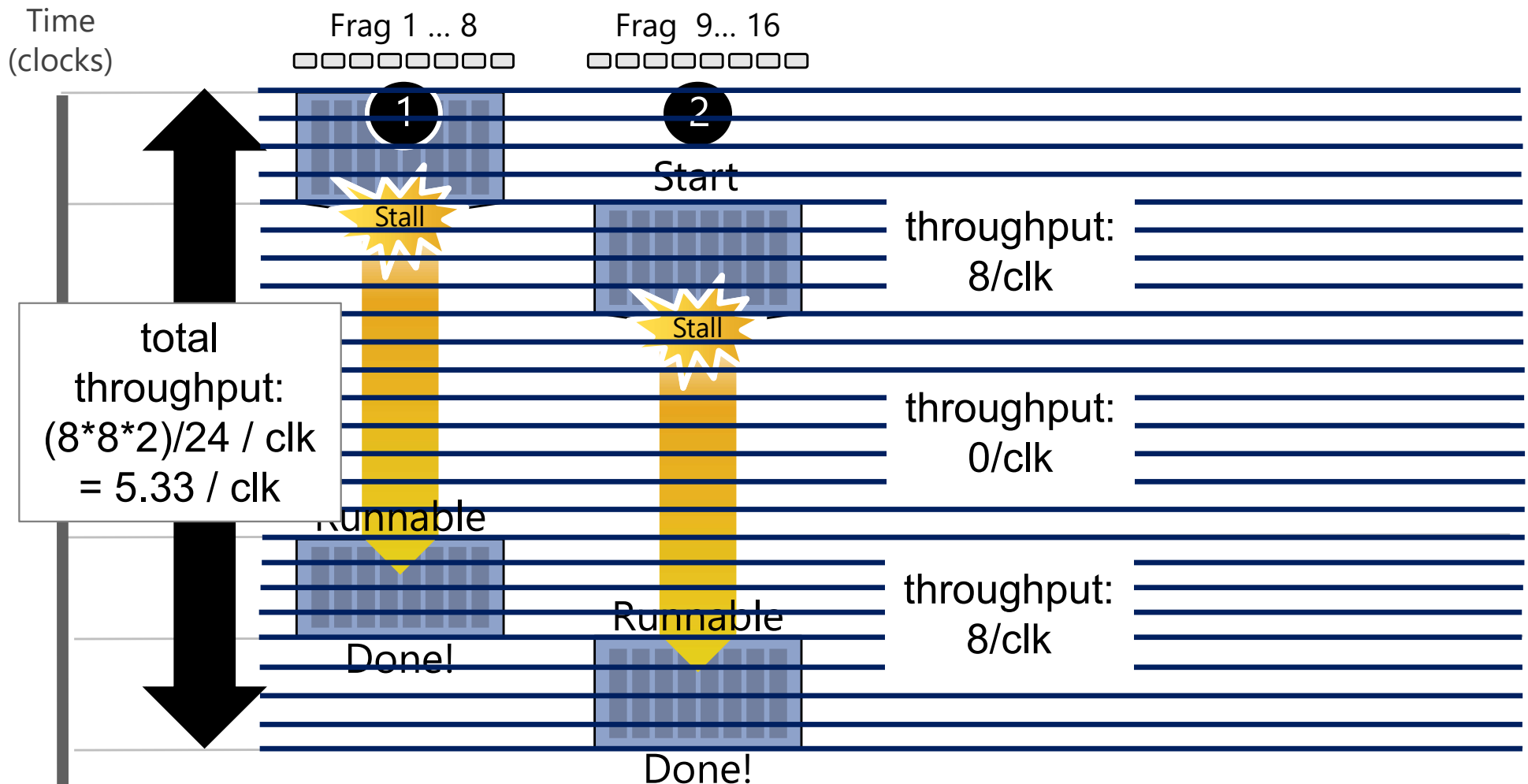
Throughput! (4 groups of threads)



Throughput! (2 groups of threads)



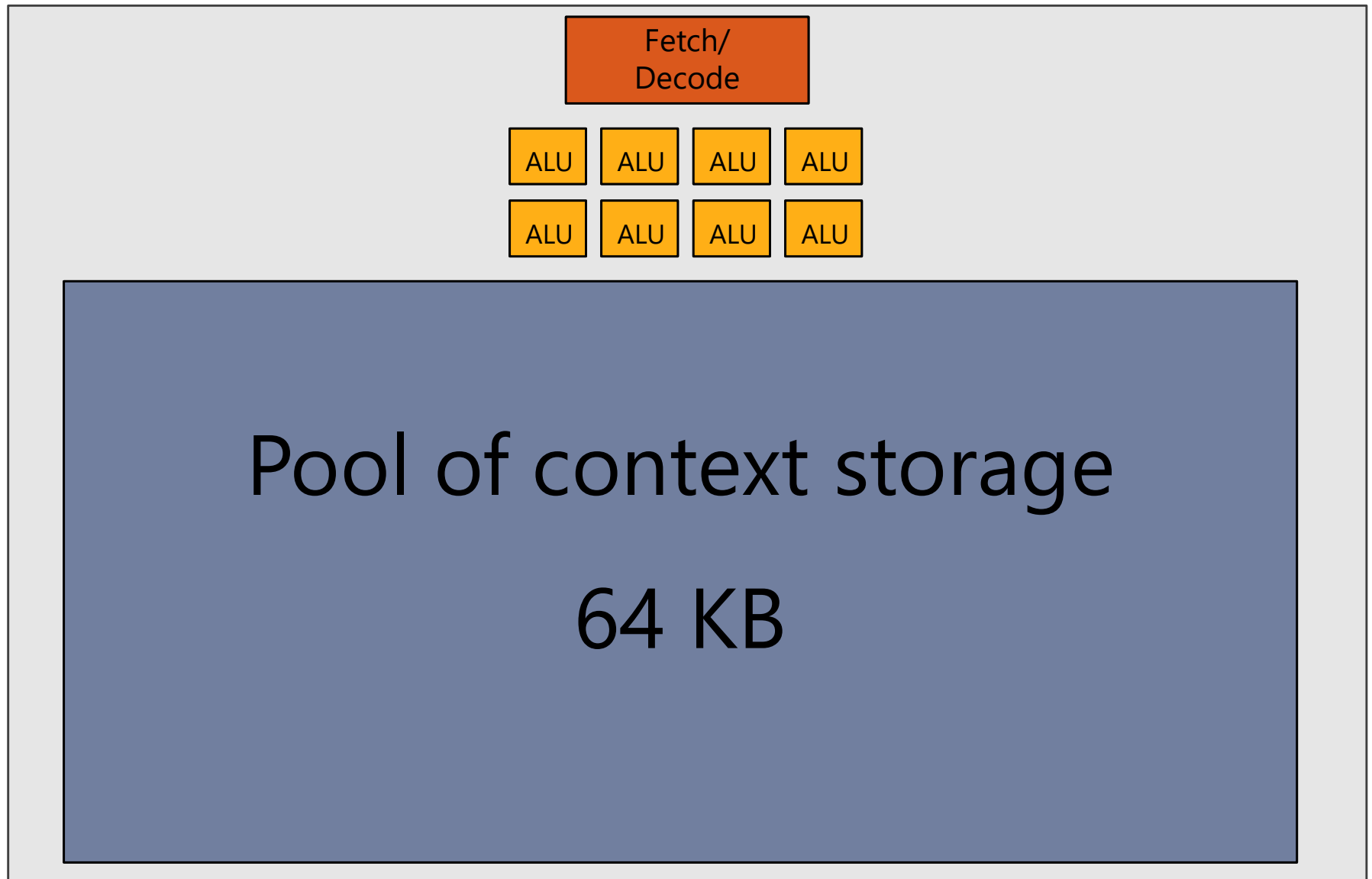
Throughput! (2 groups of threads)



why is the (average) throughput per clock $(8 \cdot 8 \cdot 2) / 24$?

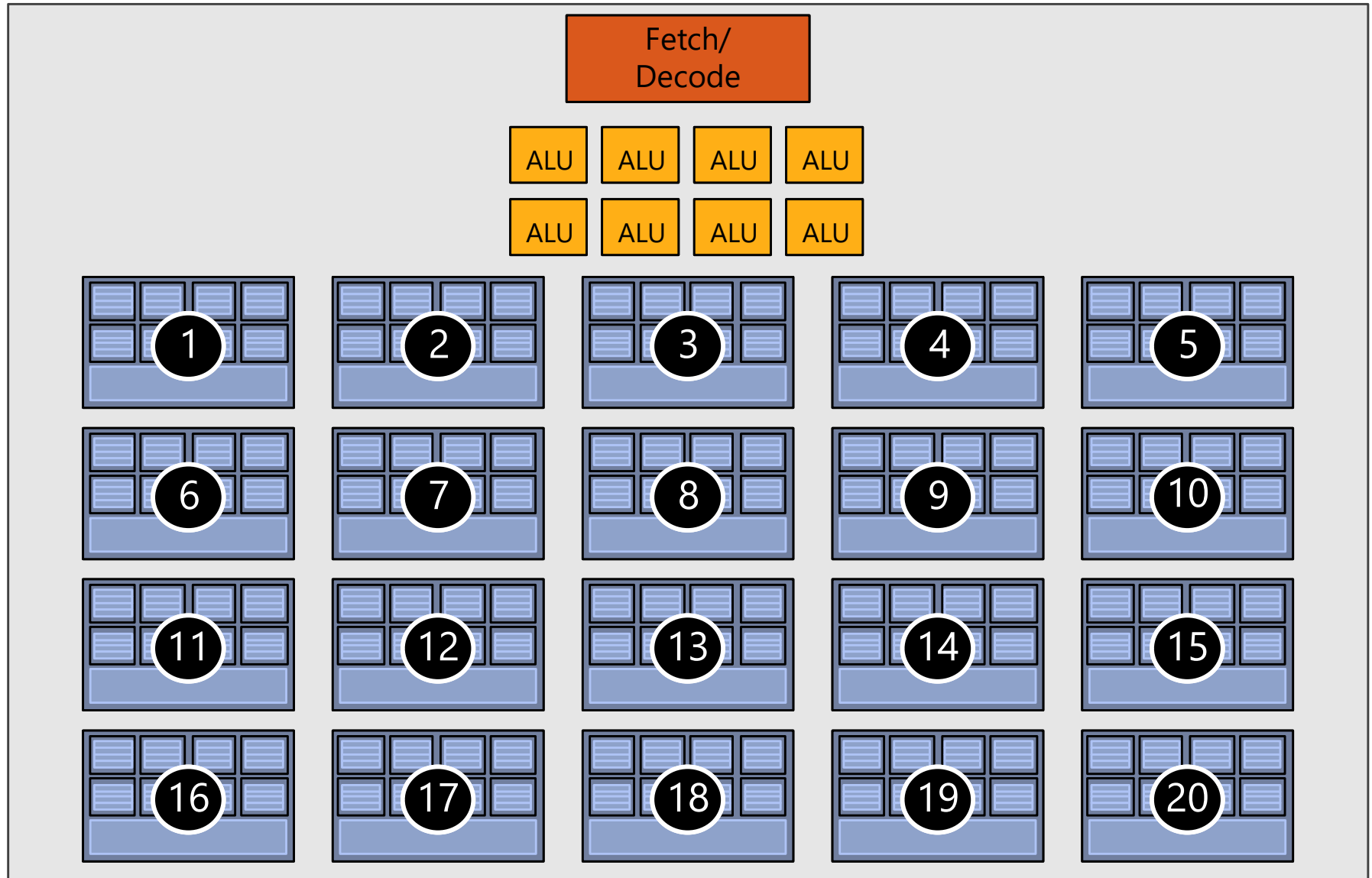
- 8 threads per group
- 8 clock cycles to completely finish one group
- 2 groups
- 24 clock cycles from start to finish (first group to last group) [instead of 16 clock cycles for peak!]

Storing contexts

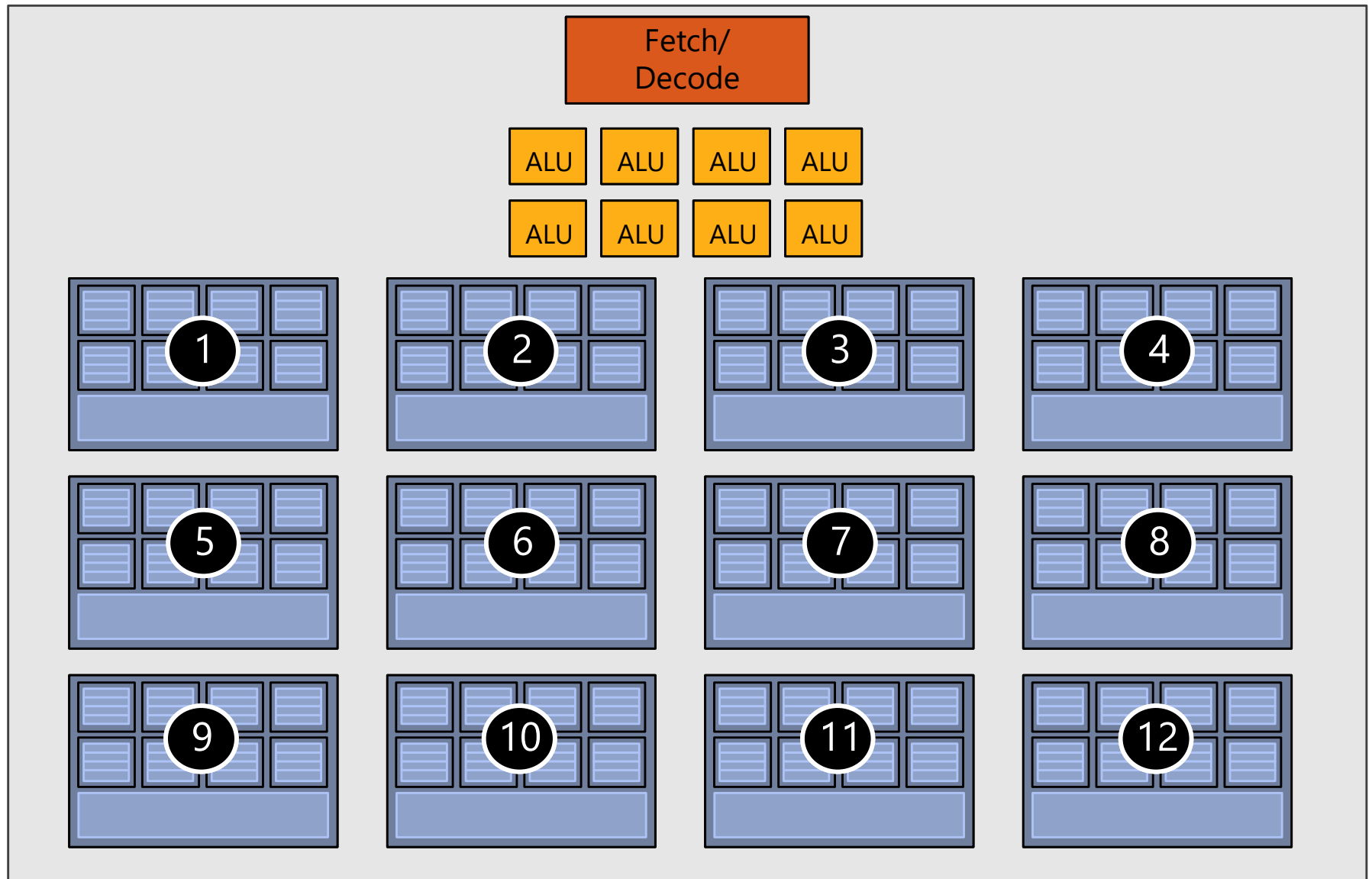


Twenty small contexts (few regs/thread)

(maximal latency hiding ability)

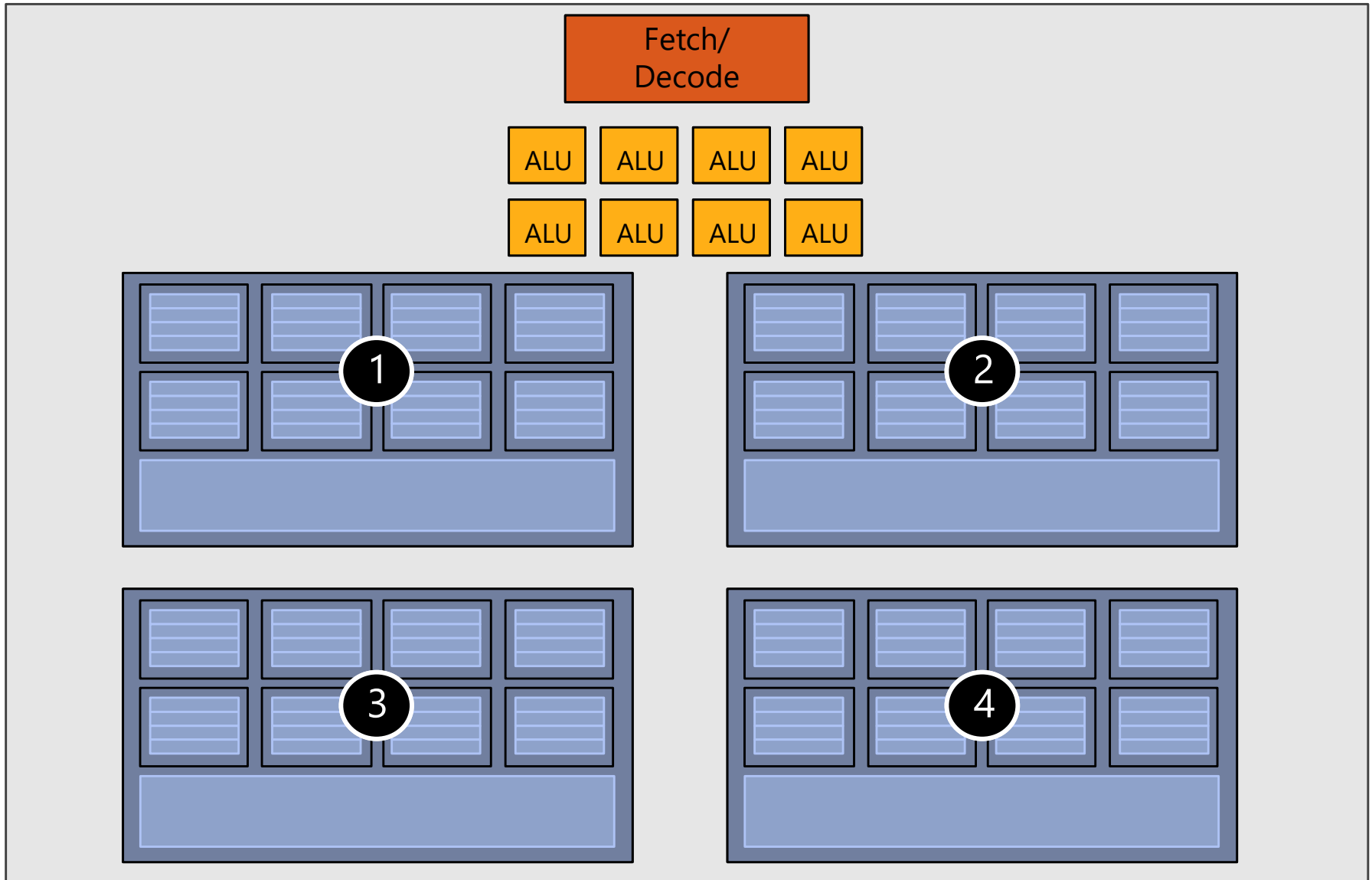


Twelve medium contexts (more regs/th.)



Four large contexts (many regs/thread)

(low latency hiding ability)



Concepts: SM Occupancy in CUDA (*TLP!*)



We need to hide latencies from

- Instruction pipelining hazards (RAW – read after write, etc.)
(also: branches; behind branch, fetch instructions from different instruction stream)
- Memory access latency

First type of latency: Definitely need to hide! (it is always there)

Second type of latency: only need to hide if it does occur (of course not unusual)

Occupancy: How close are we to *maximum latency hiding ability*?
(how many threads are resident vs. how many could be)

See run time occupancy API, or Nsight Compute: <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html#occupancy-calculator>

Complete GPU

16 cores

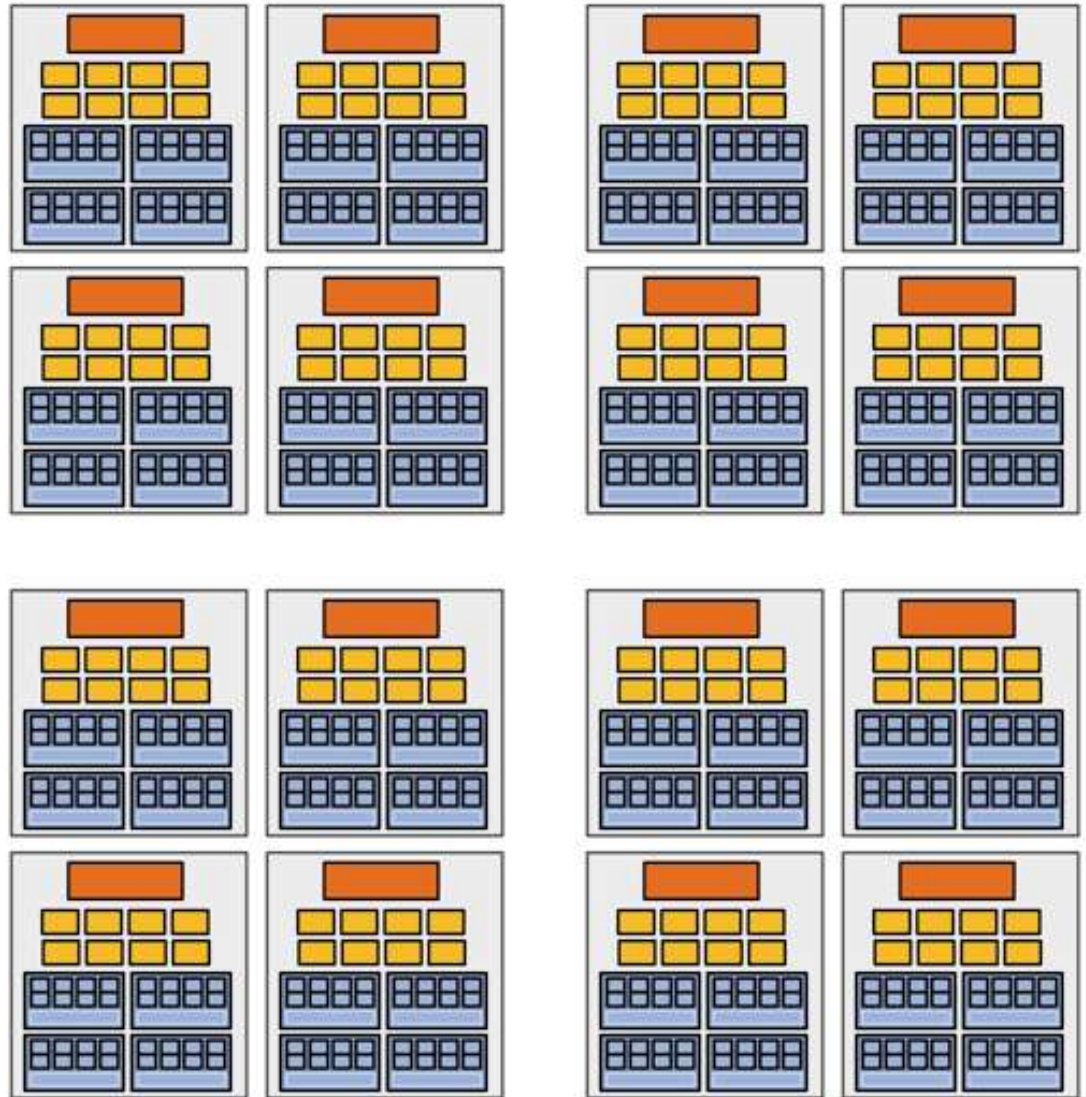
8 mul-add_[mad] ALUs per core
(8*16 = **128** total)

16 simultaneous
instruction streams

64 (4*16) concurrent (but
interleaved) instruction streams

512 (8*4*16) concurrent
fragments (resident threads)

= **256 GFLOPs** (@ 1GHz)
(**128** * 2_[mad] * 1G)



Complete GPU

16 cores

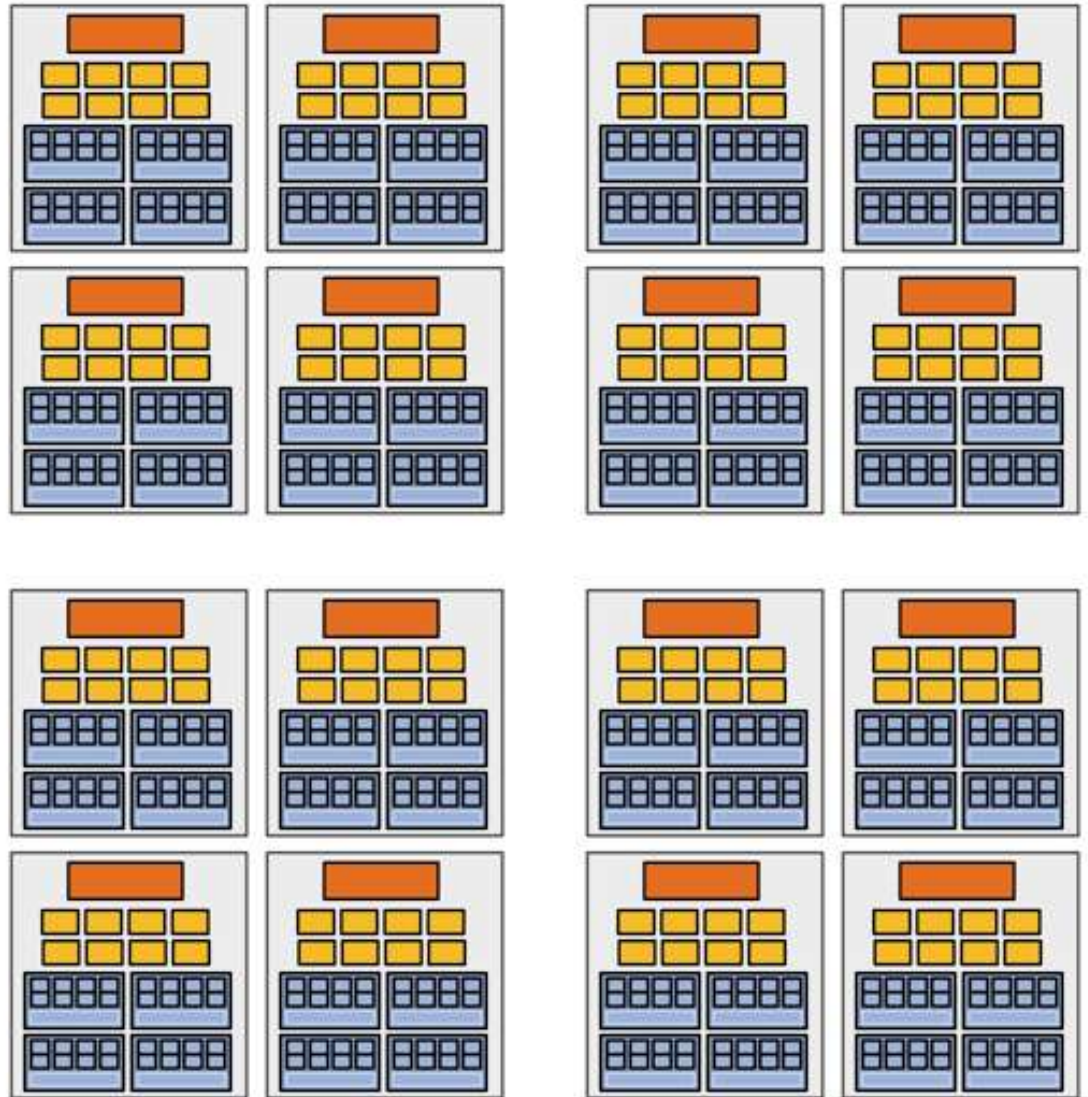
8 mul-add _[mad] ALUs per core
(8*16 = **128** total)

16 simultaneous
instruction streams

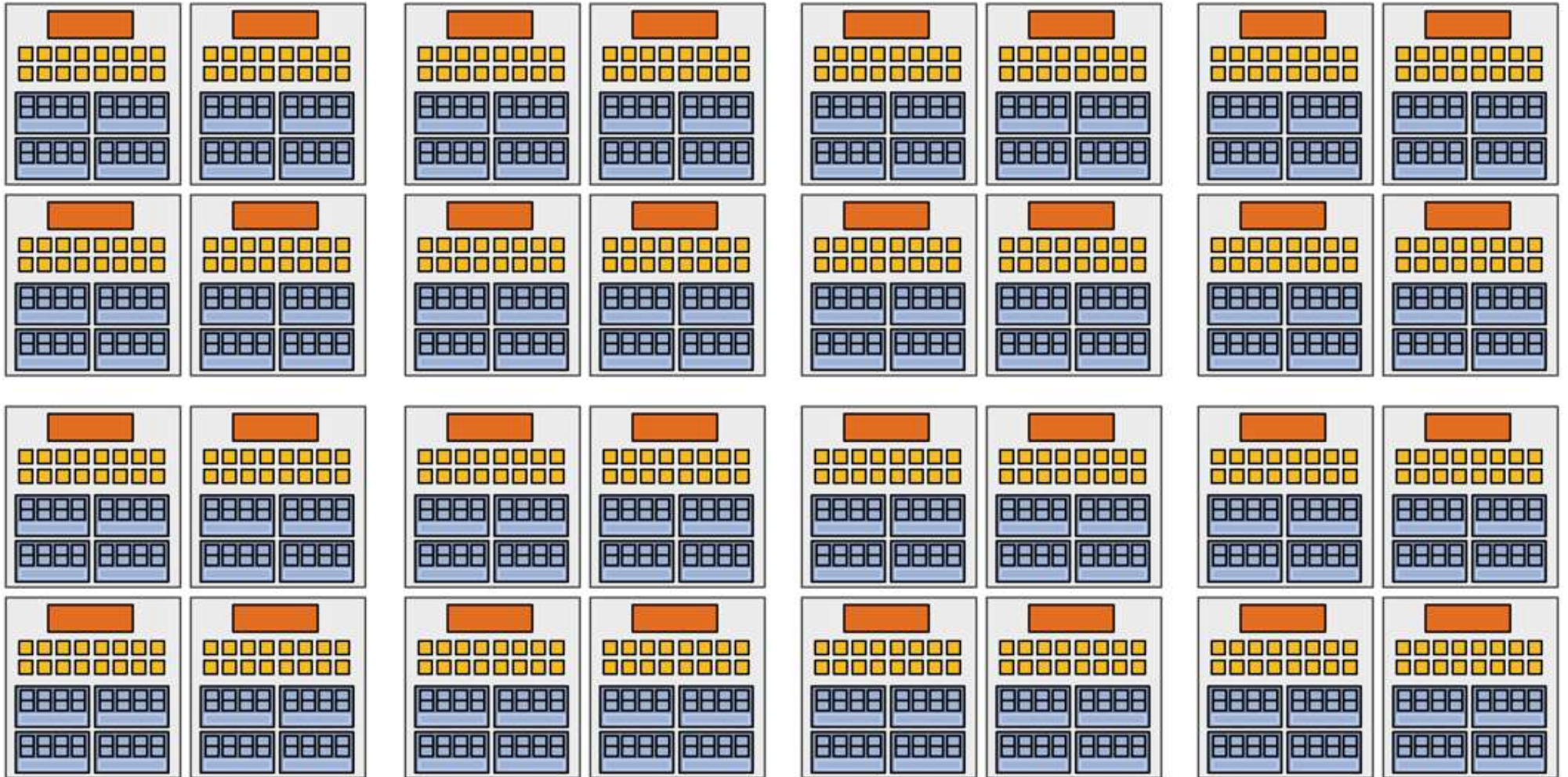
64 (4*16) concurrent (but
interleaved) instruction streams

512 (8*4*16) concurrent
fragments (resident threads)

= **256 GFLOPs** (@ 1GHz)
(**128** * 2 _[mad] * 1G)



"Enthusiast" GPU (Some time ago :)



32 cores, 16 ALUs per core (512 total) = 1 TFLOP (@ 1 GHz)

Where We've Arrived...



Summary: three key ideas for high-throughput execution

1. Use many “slimmed down cores,” run them in parallel
2. Pack cores full of ALUs (by sharing instruction stream overhead across groups of fragments)
 - Option 1: Explicit SIMD vector instructions
 - Option 2: Implicit sharing managed by hardware
3. Avoid latency stalls by interleaving execution of many groups of fragments
 - When one group stalls, work on another group

GPUs are here!
(usually)

Thank you.